LAPORAN PRAKTIKUM

STRUKTUR DATA

“IMPLEMENTASI ADT STACK”

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik Implementasi ADT *Stack*. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 15 April 2026

Hafizh Habibullah

DAFTAR ISI

KATA PENGANTAR 1
DAFTAR ISI 2
BAB I PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Tujuan 3
1.3 Manfaat 3
BAB 2 PEMBAHASAN 5
2.1 Latihan Dasar Pemahaman Stack 5
2.2 Implementasi Stack menggunakan Array 6
2.2.1 Konstruktor dan Validator 6
2.2.2 Method-Method Utama 7
2.2.3 Method Tambahan 8
2.3 Kelas Driver Stack Array 9
2.4 Postfix Menggunakan Stack 10
2.5 Nilai Maksimum Pada Stack 12
2.6 Implementasi Stack menggunakan ArrayList 13
2.6.1 Kelas Siswa 13
2.6.2 Method-Method Utama 14
2.6.3 Fungsi Main 15
2.7 Latihan Simulasi Browser History 16
2.7.1 Kelas ADT Website 16
2.7.2 Kelas Driver 17
BAB III KESIMPULAN 19
3.1 Kesimpulan 19
DAFTAR PUSTAKA 20

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu komputer, pengelolaan data tidak hanya membutuhkan struktur yang mampu menyimpan elemen secara dinamis, namun juga struktur yang mengatur urutan akses terhadap data tersebut. Salah satu struktur data yang menerapkan aturan akses secara ketat adalah Stack atau tumpukan.

Stack bekerja berdasarkan prinsip LIFO (Last In First Out), yaitu elemen yang terakhir dimasukkan akan menjadi elemen yang paling pertama dikeluarkan, mirip seperti tumpukan piring yang di mana piring yang paling atas yang paling pertama diambil.

Stack merupakan salah satu implementasi dari Abstarct Data Type (ADT) yang banyak digunakan dalam berbagai aplikasi nyata, seperti mekanisme undo/redo pada aplikasi teks, penelusuran rekursi, pengecekan keseimbangan tanda kurung, hingga navigasi halaman pada browser. Dengan memahami cara kerja Stack, pengembang perangkat lunak dapat menyelesaikan berbagai permasalahan yang melibatkan pemrosesan data secara terbalik atau berbasis urutan masuk.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep ADT Stack dan prinsip kerja LIFO yang mendasarinya.
2. Mampu mengidentifikasi dan menjelaskan operasi-operasi dasar pada Stack seperti push(), pop(), peek(), dan isEmpty().
3. Mampu mengimplementasikan ADT Stack menggunakan bahasa pemrograman Java.
4. Memahami perbedaan antara Stack dan struktur data lainnya seperti ArrayList dalam hal aturan akses elemen.

1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Menerapkan konsep ADT Stack dalam menyelesaikan permasalahan pemrograman yang membutuhkan mekanisme LIFO.

2. Memberikan pemahaman yang lebih luas terkait berbagai jenis struktur data dan kapan masing-masing struktur tersebut tepat untuk digunakan.
3. Mengimplementasikan Stack secara mandiri sebagai fondasi untuk mempelajari struktur data yang lebih kompleks seperti Queue dan Tree.

BAB 2
PEMBAHASAN

2.1 Latihan Dasar Pemahaman Stack

```java
package pekan3_2511531002;

import java.util.Stack;
public class LatihanStack_2511531002 {
    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(42);
        s.push(-3);
        s.push(17);
        System.out.println("nilai stack = " + s);
        System.out.println("nilai pop = " + s.pop());
        System.out.println("nilai stack setelah pop = " + s);
        System.out.println("nilai peek = " + s.peek());
        System.out.println("nilai stack setelah peek = " + s);
    }
}
```

Kode Program 2. 1 – Kelas Latihan Stack

Sebagai permulaan praktikum, sebuah kelas diberikan sebagai pemahaman dasar terkait ADT Stack. Pada kasus ini, dideklarasikan sebuah stack dengan tipe Integer dengan nama “s”. Kemudian, dimasukkan nilai 42, -3, dan 17 secara berurutan menggunakan method push(). Lalu, isi dari stack tersebut akan dicetak.

Stack menerapkan Prinsip LIFO, yang artinya data yang terakhir kali masuk adalah data yang pertama kali dapat dikeluarkan. Karena 17 adalah data terakhir yang dimasukkan, maka ia terletak pada posisi paling atas.

Setelah itu digunakan method pop(). Method ini akan mengembalikan sekaligus menghapus nilai paling atas dari suatu stack. Lalu, digunakan method peek() yang berguna untuk mengembalikan nilai paling atas dari suatu stack. Karena 17 sudah dihilangkan menggunakan pop(), maka nilai yang paling atas saat ini adalah -3. Maka method peek() akan mengembalikan nilai -3 saat dipanggil.

Terakhir, dicetak kembali isi dari stack setelah penghapusan nilai, yang dimana hasil dari program tersebut ketika dijalankan akan menjadi seperti berikut:

```
Console ×   Progress   Git Staging
<terminated> LatihanStack_2511531002 [Java App
nilai stack = [42, -3, 17]
nilai pop = 17
nilai stack setelah pop = [42, -3]
nilai peek = -3
nilai stack setelah peek = [42, -3]
```

Gambar 2. 1 – Output dari Kode Program Latihan Stack

2.2 Implementasi Stack menggunakan Array

2.2.1 Konstruktor dan Validator

```java
package pekan3_2511531002;

public class StackArray_2511531002 {
    static final int MAX = 1000;
    int top_2511531002;
    int a[] = new int[MAX];
    boolean isEmpty()
    {
        return (top_2511531002< 0);
    }
    StackArray_2511531002()
    {
        top_2511531002 = -1;
    }
```

Kode Program 2. 2 – Blok Program Konstruktor dan Validator

Class ini mengimplementasikan Stack secara manual menggunakan Array statis berukuran MAX = 1000. Atribut top berfungsi sebgaia penunjuk indeks elemen teratas stack dan diinisialisasi dengan nilai -1 di konstruktor sebagai penanda bahwa stack masih kosong.

Memeriksa apakah stack kosong degnan memeriksa apakah top < 0. Karena top dimulai dari -1 dan bertambah setiap push, kondisi ini hanya akan terpenuhi ketika belum ada elemen sama sekali atau semua elemen sudah di-pop.

2.2.2 Method-Method Utama

```
boolean push_2511531002(int x)
{
    if (top_2511531002 ≥ (MAX - 1)) {
        System.out.println("Stack Overflow");
        return false;
    }
    else {
        a[++top_2511531002] = x;
        System.out.println(x + " dimasukkan dalam stack");
        return true;
    }
}
int pop_2511531002()
{
    if (top_2511531002 < 0) {
        System.out.println("Stack Underflow");
        return 0;
    }
    else {
        int x = a[top_2511531002--];
        return x;
    }
}
```

Kode Program 2. 3 – Blok Program Push and Pop

Method push() memastikan sebelum memasukkan elemen, diperiksa dulu apakah top >= MAX-1. Jika iya, berarti stack sudah penuh dan terjadi **Stack Overflow**. Jika masih ada ruang, top di-increment terlebih dahulu lalu kemudian elemen x disimpan di a[top], sehingga elemen baru selalu menempati posisi di atas elemen sebelumnya.

Method pop() memeriksa terlebih dahulu apakah top < 0. Jika iya, berarti stack kosong dan terjadi **Stack Underflow**. Jika tidak, elemen teratas disimpan terlebih dahulu ke variabel x, kemudian top di-decrement, lalu x dikembalikan. Elemen lamanya di Array sebenarnya masih ada secara fisik, namun sudah tidak bisa diakses lagi karena top sudah turun.

```
int peek_2511531002()
{
    if (top_2511531002 < 0) {
        System.out.println("Stack Underflow");
        return 0;
    }
    else {
        int x = a[top_2511531002];
        return x;
    }
}
```

Kode Program 2. 4 – Blok Program Peek

Sama seperti pop dalam hal pemeriksaan underflow, bedanya peek hanya mengambil nilai a[top] tanpa mengubah nilai top sama sekali, sehingga isi stack tidak berubah.

2.2.3 Method Tambahan

```
    void print() {
        for(int i = top_2511531002; i > -1; i--) {
            System.out.print(" " + a[i]);
        }
    }
}
```

Kode Program 2. 5 – Blok Program Print

Mencetak isi stack dari elemen teratas ke elemen terbawah dengan iterasi mundur dari indeks top sampai 0. Ini mencerminkan urutan LIFO secara visual, yaitu elemen yang paling terakhir masuk yang ditampilkan paling atas.

2.3 Kelas Driver Stack Array

```java
package pekan3_2511531002;

public class StackArrayDriver_2511531002 {
    public static void main(String[] args) {
        StackArray_2511531002 s = new StackArray_2511531002();
        s.push_2511531002(10);
        s.push_2511531002(20);
        s.push_2511531002(30);
        System.out.println(s.pop_2511531002() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah : " + s.peek_2511531002());
        System.out.print("Elemen pada stack : ");
        s.print();
    }
}
```

Kode Program 2. 6 – Program Driver untuk Stack Array

Objek stack dibuat dari class stackArray yang sebelumnya sudah dibuat, lalu tiga elemen dimasukkan berurutan 10, 20, 30; sehingga 30 beradai di paling atas. Kemudian pop() dipanggil yang mengambil dan menghapus elemen teratas, yaitu 30, lalu peek() mengintip elemen teratas yang sekarang adalah 20 tanpa menghapusnya. Terakhir, print() mencetak sisa isi stack dari atas ke bawah, yaitu 20 lalu 10. Hasil Outputnya adalah sebagai berikut:

```
Console   Progress   Git Staging
<terminated> StackArrayDriver_2511531002 [Jav
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah : 20
Elemen pada stack :  20 10
```

Gambar 2. 2 – Hasil Output dari Program Driver Stack Array

2.4 Postfix Menggunakan Stack

```java
package pekan3_2511531002;
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511531002 {
    public static int postFixEvaluate_2511531002(String expression) {
        Stack<Integer> s  = new Stack<Integer>();
        Scanner input = new Scanner(expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) {
                if(!s.isEmpty()) {
                    System.out.println(s);
                }
                s.push(input.nextInt());
            } else {
                if(!s.isEmpty()) {
                    System.out.println(s);
                }
                String operator = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator.equals("+")) {
                    s.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2);
                }
            }
        }
        input.close();
        return s.pop();
    }
    public static void main(String[] args) {
        System.out.println("hasil postfix = " + postFixEvaluate_2511531002("5 2 4 * + 7 -"));
    }
}
```

Kode Program 2. 7 – Program Postfix

Postfix adalah cara penulisan ekspresi matematika dimana operator diletakkan setelah operand-nya. Misalkan 5 + 2 yang dimana di dalam postfix akan menjadi 5 2 +.

Method postfixEvaluate() menerima ekspresi postfix dalam bentuk String. Lalu, dibuat sebuah Stack untuk menyimpan angka, dan Scanner untuk membaca angka/operator dari String tersebut satu per satu yang dipisahkan dengan spasi.

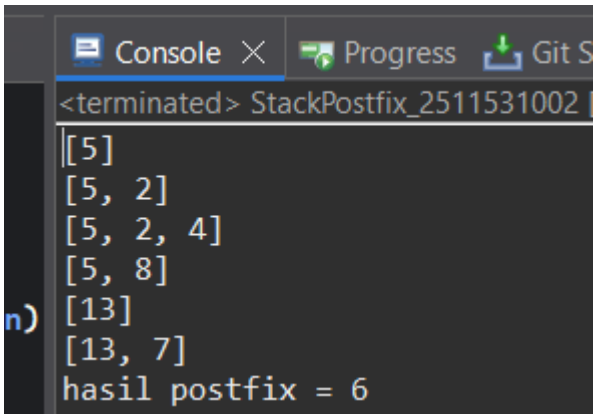
Jika karakter yang dibaca adalah angka (hasNextInt()), maka akan langsung di-push ke Stack. Sebelum push, isi stack dicetak terlebih dahulu jika tidak kosong.

Jika karakter yang dibaca bukanlah angka, berarti ia merupakan operator. Dua elemen teratas di-pop sebagai operand2 dan operand1. Kemudian, operasi

dijalankan sesuai dengan operator yang didapatkan, antara (+, -, *) menggunakan operand2 dan operand1.

Setelah semua karakter habis, tersisa satu elemen di stack yaitu hasil akhir, yang nantinya akan di-pop dan dikembalikan.

Misalkan diberikan masukan String (“5 2 4 * + 7 –“), hasil serta proses sederhananya adalah sebagai berikut:



Gambar 2. 3 – Hasil Percobaan Postfix

2.5 Nilai Maksimum Pada Stack

```java
package pekan3_2511531002;
import java.util.Stack;

public class NilaiMaksimum_2511531002 {
    public static int max(Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer>();
        int maxValue = s.pop();
        backup.push(maxValue);
        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            maxValue = Math.max(maxValue, next);
        }
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return maxValue;
    }
    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack = " + s);
        System.out.println("Stack Teratas = " + s.peek());
        System.out.println("Nilai maksimum = " + max(s));
    }
}
```

Kode Program 2. 8 – Program Nilai Maksimum menggunakan Stack

Karena stack tidak bisa diakses secara acak seperti array, satu-satunya cara untuk membaca semua elemennya adalah dengan pop() satu per satu. Tapi masalahnya adalah pop() merusak isi stack asli. Karena itu, dibutuhkan stack cadangan (backup) untuk menampung elemen yang sudah di-pop agar bisa dikembalikan lagi nantinya.

Elemen pertama yang di-pop langsung dijadikan nilai asumsi awal maxValue. Lalu, selama stack belum kosong, tiap elemen di-pop, ditampung ke backup, dan dibandingkan dengan maxValue menggunakan Math.max(). Setelah semua elemen habis dibaca, isi backup di-pop kembali satu per satu dan di-push kedalam stack s semula. Karena backup urutannya terbalik, hasil push balik ke s akan mengembalikan urutan stack ke kondisi semula. Hasil output dari contoh program maksimum stack adalah sebagai berikut:

```
Console ×   Progress   Gr
<terminated> NilaiMaksimum_251153
isi stack = [70, 12, 20]
Stack Teratas = 20
Nilai maksimum = 70
```

Gambar 2. 4 – Hasil Program Maksimum Stack

2.6 Implementasi Stack menggunakan ArrayList

2.6.1 Kelas Siswa

```java
class Siswa_2511531002 {
    String nama;
    int nim;

    public Siswa_2511531002(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "Nim: " + nim + ", Nama: " + nama;
    }
}
```

Kode Program 2. 9 – Kelas Siswa

Class sederhana dengan atribut nama dan nim, constructor untuk menginisialisasi keduanya, dan toString() yang di-override agar objek Siswa bisa dicetak dengan format yang rapi.

2.6.2 Method-Method Utama

```java
public class SiswaStack_2511531002 {
    private ArrayList<Siswa_2511531002> stack;

    public SiswaStack_2511531002() {
        stack = new ArrayList<>();
    }

    public void push(Siswa_2511531002 mhs) {
        stack.add(mhs);
    }

    public Siswa_2511531002 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswa_2511531002 peek() {
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa_2511531002() {
        for (int i = stack.size() - 1; i ≥ 0; i--) {
            System.out.println(stack.get(i));
        }
    }
```

Kode Program 2. 10 – Implementasi Kelas Siswa menggunakan ArrayList

Kelas ini memiliki konstruktor, method-method umum pada stack, dan method tambahan yang berupa menampilkan semua isi dari stack. Stack tidak diimplementasikan dengan array atau class Stack bawaan Java, melainkan menggunakan *ArrayList*. Elemen teratas Stack direpresentasikan sebagai elemen terakhir di ArrayList.

Karena stack saat ini diimplementasikan menggunakan ArrayList, untuk memasukkan elemen ke dalam stack, digunakan method .add().

Untuk mengambil dan mengembalikan elemen terakhir pada ArrayList, digunakan method remove(). Sebelum itu, diperiksa terlebih dahulu apakah stack kosong atau tidak. Jika kosong, maka nilai yang akan dikembalikan adalah “null”.

Untuk mengambil nilai terakhir pada stack, pastikan apakah stack kosong atau tidak. Jika tidak, nilai bisa diambil menggunakan method .get() tanpa harus menghapus nilai terakhir. Jika stack kosong, akan dikembalikan nilai “null”.

Validator pada kelas ini sederhana, hanya wrapper dari method isEmpty() ArrayList.

Terakhir adalah method untuk menampilkan isi elemen dengan cara iterasi mundur, dari elemen terakhir ke elemen pertama. Sehingga elemen yang dicetak pertama adalah elemen teratas stack.

2.6.3 Fungsi Main

```
52      public static void main(String[] args) {
53          SiswaStack_2511531002 studentStack_2511531002 = new SiswaStack_2511531002();
54
55          Siswa_2511531002 mhs1 = new Siswa_2511531002("Ali", 1);
56          Siswa_2511531002 mhs2 = new Siswa_2511531002("Boby", 2);
57          Siswa_2511531002 mhs3 = new Siswa_2511531002("Charles", 3);
58
59          studentStack_2511531002.push(mhs1);
60          studentStack_2511531002.push(mhs2);
61          studentStack_2511531002.push(mhs3);
62
63          System.out.println("Siswa di dalam stack: ");
64          studentStack_2511531002.tampilkanSiswa_2511531002();
65
66          System.out.println("siswa teratas: " + studentStack_2511531002.peek());
67          System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack_2511531002.pop());
68          System.out.println("daftar siswa setelah di pop: ");
69          studentStack_2511531002.tampilkanSiswa_2511531002();
70      }
71  }
```

Kode Program 2. 11 – Fungsi Main

Tiga objek Siswa dibuat dan di-push secara berurutan, sehingga Charles berada di paling atas. Setelah semua dicetak, peek() akan mengintip Charles, lalu pop() akan mengeluarkannya, dan stack akan dicetak ulang

menyisakan Ali dan Boby. Hasil Output dari program tersebut adalah sebagai berikut:

```
Console   Progress   Git Staging
<terminated> SiswaStack_2511531002 [Java Application] C:\Users\user\.p2\pool\plugins\
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
siswa teratas: Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
daftar siswa setelah di pop:
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

Gambar 2.5 – Contoh Hasil Output Program Stack Siswa

2.7 Latihan Simulasi Browser History

2.7.1 Kelas ADT Website

```java
package pekan3_2511531002;

public class Website_2511531002 {
	private String judul_1002;
	private String url_1002;

	public Website_2511531002(String judul_1002, String
url_1002) {
		this.judul_1002 = judul_1002;
		this.url_1002 = url_1002;
	}

	public String getJudul_1002() { return judul_1002; }
	public String getUrl_1002() { return url_1002; }

	public void setJudul_1002(String judul_1002) {
this.judul_1002 = judul_1002; }
	public void setUrl_1002(String url_1002) { this.url_1002 =
url_1002; }

	@Override
	public String toString() {
		return "Judul : " + judul_1002 + "\nURL   : " +
url_1002;
	}
}
```

Kode Program 2. 12 – Kelas Website_2511531002

2.7.2 Kelas Driver

```java
package pekan3_2511531002;

import java.util.Scanner;
import java.util.Stack;
public class Browser_2511531002 {
        //  0. Tampilkan Menu
        public static void tampilkanMenu_2511531002() {
                System.out.println("\n=== Browser History NIM: 2511531002 ===");
                System.out.println("1. Kunjungi Website (Push)");
                System.out.println("2. Tombol Back (Pop)");
                System.out.println("3. Lihat Halaman Aktif (Peek)");
                System.out.println("4. Keluar");
        }

        // 1. Kunjungi Website (Push)
        public static void kunjungiWebsite_2511531002(Stack<Website_2511531002>
stack, Scanner sc) {
                System.out.print("\nMasukkan Judul: ");
                String judul_1002 = sc.nextLine();
                System.out.print("Masukkan URL: ");
                String url_1002 = sc.nextLine();
                Website_2511531002 web_1002 = new Website_2511531002(judul_1002,
url_1002);
                stack.push(web_1002);
                System.out.println("Berhasil mengunjungi halaman!");
        }

        // 2. Tombol Back (Pop)
        public static void tombolBack_2511531002(Stack<Website_2511531002> stack)
{
                if (!stack.isEmpty()) {
                        Website_2511531002 hapus_1002 = stack.pop();
                        System.out.println("Kembali dari: " +
hapus_1002.getJudul_1002());
                } else {
                        System.out.println("History kosong! Tidak bisa kembali.");
                }
        }

        // 3. Lihat Halaman Aktif (Peek)
        public static void lihatHalAktif_2511531002(Stack<Website_2511531002>
stack) {
                if (!stack.isEmpty()) {
                        System.out.println("\nHalaman aktif:");
                        System.out.println(stack.peek());
                } else {
                        System.out.println("Tidak ada halaman aktif.");
                }
        }

        public static void main(String[] args) {
                Stack<Website_2511531002> history_1002 = new Stack<>();
                Scanner sc = new Scanner(System.in);
                int pilihan_1002;

                do {
                        tampilkanMenu_2511531002();
                        System.out.print("Pilih menu: ");
                        pilihan_1002 = sc.nextInt();
                        sc.nextLine();

                        switch (pilihan_1002) {
                        case 1:
                                kunjungiWebsite_2511531002(history_1002, sc);
                                break;
                        case 2:
                                tombolBack_2511531002(history_1002);
                                break;
                        case 3:
```

```
                                lihatHalAktif_2511531002(history_1002);
                                break;
                        case 4:
                                System.out.println("Keluar dari program.");
                                break;
                        default:
                                System.out.println("Pilihan tidak valid.");
                        }
                } while (pilihan_1002 != 4);
                sc.close();
        }
}
```

Kode Program 2. 13 – Kelas Browser_2511531002

BAB III
KESIMPULAN

3.1 Kesimpulan

Stack adalah struktur data yang bekerja berdasarkan prinsip LIFO (Last In First Out), dimana elemen yang terakhir dimasukkan akan menjadi elemen yang paling pertama dikeluarkan. Operasi data Stack meliputi push() untuk memasukkan elemen, pop() untuk mengeluarkan elemen teratas, peek() untuk melihat elemen teratas tanpa menghapusnya, serta isEmpty() untuk mengecek apakah Stack dalam kondisi kosong atau tidak.

Praktikum ini menunjukkan bahwa Stack dapat diimplementasikan dengan berbagai pendekatan menggunakan class Stack bawaan Java, menggunakan Array Statis secara manual, maupun menggunakan ArrayList sebagai backing strucutre. Ketiga pendekatan menghasilkan perilaku yang sama selama operasi push dan pop mengikuti aturan LIFO, yang membuktikan bahwa Stack adalah konsep abstrak yang tidak bergantung pada satu jenis implementasi tertentu.

Selain itu, Stack terbukti berguna dalam menyelesaikan permasalahan nyata seperti evaluasi ekspresi postfix, dimana operand ditumpuk ke stack dan diproses ketika operator ditemukan, serta pencarian nilai maksimum dalam stack yang membutuhkan stack tambahan sebagai backup agar isi stack asli tetap terjaga setelah proses selesai. Secara keseluruhan, pemahaman terhadap ADT Stack memberikan dasar yang kuat untuk mempelajari struktur data lanjutan seperti Queue dan Tree.

DAFTAR PUSTAKA

[1] Modul Praktikum Struktur Data.

[2] Oracle. *Java Documentation.* https://docs.oracle.com/javase/